

这样产生出来的代码又大又慢, 而且基础不稳, 因为每次 window class 继承体系一有改变, 所有这一类代码都必须再次检阅看看是否需要修改。例如一旦加入新的 derived class, 或许上述连串判断中需要加入新的条件分支。这样的代码应该总是以某些“基于 virtual 函数调用”的东西取而代之。

优良的 C++ 代码很少使用转型, 但若说要完全摆脱它们又太过不切实际。例如 p.118 从 int 转型为 double 就是转型的一个通情达理的使用, 虽然它并非绝对必要(那段代码可以重新写过, 声明一个类型为 double 的新变量并以 x 值初始化)。就像面对众多蹊跷可疑的构造函数一样, 我们应该尽可能隔离转型动作, 通常是把它隐藏在某个函数内, 函数的接口会保护调用者不受函数内部任何肮脏龌龊的动作影响。

请记住

- 如果可以, 尽量避免转型, 特别是在注重效率的代码中避免 dynamic_casts。如果有个设计需要转型动作, 试着发展无需转型的替代设计。
- 如果转型是必要的, 试着将它隐藏于某个函数背后。客户随后可以调用该函数, 而不需将转型放进他们自己的代码内。
- 宁可使用 C++-style (新式) 转型, 不要使用旧式转型。前者很容易辨识出来, 而且也比较有着分门别类的职掌。

条款 28: 避免返回 handles 指向对象内部成分

Avoid returning "handles" to object internals.

假设你的程序涉及矩形。每个矩形由其左上角和右下角表示。为了让一个 Rectangle 对象尽可能小, 你可能会决定不把定义矩形的这些点存放在 Rectangle 对象内, 而是放在一个辅助的 struct 内再让 Rectangle 去指它:

```
class Point {           //这个 class 用来表述“点”
public:
    Point(int x, int y);
    ...
    void setX(int newVal);
    void setY(int newVal);
    ...
};
```

```

struct RectData {           //这些“点”数据用来表现一个矩形
    Point ulhc;              //ulhc="upper left-hand corner" (左上角)
    Point lrhc;              //lrhc="lower right-hand corner" (右下角)
};

class Rectangle {
    ...
private:
    std::tr1::shared_ptr<RectData> pData; //关于 tr1::shared_ptr,
};                                     //见条款 13

```

Rectangle 的客户必须能够计算 Rectangle 的范围，所以这个 class 提供 upperLeft 函数和 lowerRight 函数。Point 是个用户自定义类型，所以根据条款 20 给我们的忠告（它说以 *by reference* 方式传递用户自定义类型往往比以 *by value* 方式传递更高效），这些函数于是返回 references，代表底层的 Point 对象：

```

class Rectangle {
public:
    ...
    Point& upperLeft( ) const { return pData->ulhc; }
    Point& lowerRight( ) const { return pData->lrhc; }
    ...
};

```

这样的设计可通过编译，但却是错误的。实际上它是自我矛盾的。一方面 upperLeft 和 lowerRight 被声明为 const 成员函数，因为它们的目的只是为了提供客户一个得知 Rectangle 相关坐标点的方法，而不是让客户修改 Rectangle（见条款 3）。另一方面两个函数却都返回 references 指向 private 内部数据，调用者于是可通过这些 references 更改内部数据！例如：

```

Point coord1(0, 0);
Point coord2(100, 100);
const Rectangle rec(coord1, coord2);           //rec 是个 const 矩形,
                                                // 从(0,0)到(100,100)
rec.upperLeft( ).setX(50);                     //现在 rec 却变成
                                                // 从(50,0)到(100,100)

```

这里请注意，upperLeft 的调用者能够使用被返回的 reference（指向 rec 内部的 Point 成员变量）来更改成员。但 rec 其实应该是不可变的（const）！

这立刻带给我们两个教训。第一，成员变量的封装性最多只等于“返回其 reference”的函数的访问级别。本例之中虽然 ulhc 和 lrhc 都被声明为 private，它们实际上却是 public，因为 public 函数 upperLeft 和 lowerRight 传出了它们的

references。第二, 如果 const 成员函数传出一个 reference, 后者所指数据与对象自身有关联, 而它又被存储于对象之外, 那么这个函数的调用者可以修改那笔数据。这正是 bitwise constness 的一个附带结果, 见条款 3。

上面我们所说的每件事情都是由于“成员函数返回 references”。如果它们返回的是指针或迭代器, 相同的情况还是发生, 原因也相同。References、指针和迭代器统统都是所谓的 *handles* (号码牌, 用来取得某个对象), 而返回一个“代表对象内部数据”的 handle, 随之而来的便是“降低对象封装性”的风险。同时, 一如稍早所见, 它也可能导致“虽然调用 const 成员函数却造成对象状态被更改”。

通常我们认为, 对象的“内部”就是指它的成员变量, 但其实不被公开使用的成员函数(也就是被声明为 protected 或 private 者)也是对象“内部”的一部分。因此也应该留心不要返回它们的 handles。这意味你绝对不该令成员函数返回一个指针指向“访问级别较低”的成员函数。如果你那么做, 后者的实际访问级别就会提高如同前者(访问级别较高者), 因为客户可以取得一个指针指向那个“访问级别较低”的函数, 然后通过那个指针调用它。

然而“返回指针指向某个成员函数”的情况毕竟不多见, 所以让我们把注意力收回, 专注于 Rectangle class 和它的 upperLeft 以及 lowerRight 成员函数。我们在这些函数身上遭遇的两个问题可以轻松去除, 只要对它们的返回类型加上 const 即可:

```
class Rectangle {
public:
    ...
    const Point& upperLeft( ) const { return pData->ulhc; }
    const Point& lowerRight( ) const { return pData->lrhc; }
    ...
};
```

有了这样的改变, 客户可以读取矩形的 Points, 但不能涂写它们。这意味当初声明 upperLeft 和 upperRight 为 const 不再是个谎言, 因为它们不再允许客户更改对象状态。至于封装问题, 我们总是愿意让客户看到 Rectangle 的外围 Points, 所以这里是蓄意放松封装。更重要的是这是个有限度的放松: 这些函数只让渡读取权。涂写权仍然是被禁止的。

但即使如此, upperLeft 和 lowerRight 还是返回了“代表对象内部”的 handles, 有可能在其他场合带来问题。更明确地说, 它可能导致 *dangling handles* (空悬的号

码牌)：这种 *handles* 所指东西 (的所属对象) 不复存在。这种 “不复存在的对象” 最常见的来源就是函数返回值。例如某个函数返回 GUI 对象的外框 (*bounding box*)，这个外框采用矩形形式：

```
class GUIObject { ... };
const Rectangle boundingBox(const GUIObject& obj); //以 by value 方式返回一个矩形
//条款 3 谈过为什么返回类型是 const
```

现在，客户有可能这么使用这个函数：

```
GUIObject* pgo; //让 pgo 指向某个 GUIObject
...
const Point* pUpperLeft = //取得一个指针指向外框左上点
    & (boundingBox(*pgo).upperLeft());
```

对 *boundingBox* 的调用获得一个新的、暂时的 *Rectangle* 对象。这个对象没有名称，所以我们权且称它为 *temp*。随后 *upperLeft* 作用于 *temp* 身上，返回一个 *reference* 指向 *temp* 的一个内部成分，更具体地说是指向一个用以标示 *temp* 的 *Points*。于是 *pUpperLeft* 指向那个 *Point* 对象。目前为止一切还好，但故事尚未结束，因为在那个语句结束之后，*boundingBox* 的返回值，也就是我们所说的 *temp*，将被销毁，而那间接导致 *temp* 内的 *Points* 析构。最终导致 *pUpperLeft* 指向一个不再存在的对象；也就是说一旦产出 *pUpperLeft* 的那个语句结束，*pUpperLeft* 也就变成空悬、虚吊 (*dangling*)！

这就是为什么函数如果“返回一个 *handle* 代表对象内部成分”总是危险的原因。不论这所谓的 *handle* 是个指针或迭代器或 *reference*，也不论这个 *handle* 是否为 *const*，也不论那个返回 *handle* 的成员函数是否为 *const*。这里的唯一关键是，有个 *handle* 被传出去了，一旦如此你就是暴露在 “*handle* 比其所指对象更长寿” 的风险下。

这并不意味你绝对不可以让成员函数返回 *handle*。有时候你必须那么做。例如 *operator[]* 就允许你 “摘采” *strings* 和 *vectors* 的个别元素，而这些 *operator[]*s 就是返回 *references* 指向 “容器内的数据” (见条款 3)，那些数据会随着容器被销毁而销毁。尽管如此，这样的函数毕竟是例外，不是常态。

请记住

- 避免返回 *handles* (包括 *references*、指针、迭代器) 指向对象内部。遵守这个条款可增加封装性，帮助 *const* 成员函数的行为像个 *const*，并将发生 “虚吊号码牌” (*dangling handles*) 的可能性降至最低。

条款 29：为“异常安全”而努力是值得的

Strive for exception-safe code.

异常安全性（Exception safety）有几分像是……呃……怀孕。但等等，在我们完成求偶之前，实在无法确实地谈论生育。

假设有个 class 用来表现夹带背景图案的 GUI 菜单单。这个 class 希望用于多线程环境，所以它有个互斥器（mutex）作为并发控制（concurrency control）之用：

```
class PrettyMenu {
public:
    ...
    void changeBackground(std::istream& imgSrc);    //改变背景图像
    ...
private:
    Mutex mutex;                                   //互斥器
    Image* bgImage;                                //目前的背景图像
    int imageChanges;                              //背景图像被改变的次数
};
```

下面是 PrettyMenu 的 changeBackground 函数的一个可能实现：

```
void PrettyMenu::changeBackground(std::istream& imgSrc)
{
    lock(&mutex);                                  //取得互斥器（见条款 14）
    delete bgImage;                                //摆脱旧的背景图像
    ++imageChanges;                                 //修改图像变更次数
    bgImage = new Image(imgSrc);                    //安装新的背景图像
    unlock(&mutex);                                 //释放互斥器
}
```

从“异常安全性”的观点来看，这个函数很糟。“异常安全”有两个条件，而这个函数没有满足其中任何一个条件。

当异常被抛出时，带有异常安全性的函数会：

- 不泄漏任何资源。上述代码没有做到这一点，因为一旦 "new Image(imgSrc)" 导致异常，对 unlock 的调用就绝不会执行，于是互斥器就永远被把持住了。
- 不允许数据损坏。如果 "new Image(imgSrc)" 抛出异常，bgImage 就是指向一个已被删除的对象，imageChanges 也已被累加，而其实并没有新的图像被成功安装起来。（但从另一个角度说，旧图像已被消除，所以你可能会争辩说图像